

Member 1:

- **Coding (Task 2):** Implement Composite by creating a genuine recursive part-whole hierarchy with meaningful leaf and grouping behaviour.
- **Coding (Task 2):** Implement Iterator to ensure traversal without exposing the hierarchy's internal representation to client code.
- **Documentation (Task 4):** Create one of the three substantial UML Activity Diagrams.
- **Documentation (Task 1):** Co-create the complete UML class diagram drawn from scratch. (everyone adds own parts)
- **Tooling (Task 5 & Section 3, Rule 5):** Provide Valgrind evidence.
- **Tooling (Task 6):** Write the [README.md](#)

Member 2:

- **Coding (Task 2):** Implement State to handle state-dependent behaviour with valid and invalid transitions.
- **Documentation (Task 4):** Create one UML state diagram showing the lifecycle of an important stateful object.
- **Documentation (Task 4):** Create one of the three substantial UML Activity Diagrams.
- **Documentation (Task 1):** Co-create the complete UML class diagram drawn from scratch. (everyone adds own parts)
- **Tooling (Task 6 & Section 3, Rule 2):** Write a working Makefile configured to Compile using `-std=c++11`.
- **Tooling (Task 5):** Document one genuine bug encountered during development, with its symptom, cause, debugging evidence and correction and provide evidence of using GDB on a meaningful part of your program.
- In charge of the final PDF

Member 3:

- **Coding (Task 2):** Implement Decorator by building at least two meaningful decorators that can be applied at runtime, including a case where decorators are stacked.
- **Coding (Task 3):** Write main.cpp to execute "at least two non-trivial runtime scenarios" that collaborate across the patterns.
- **Design (Task 3):** Decide and document how an "existing traversal behaves when the structure it refers to changes.
- **Task 4:** one UML object diagram showing a meaningful runtime portion of nested hierarchy
- **Documentation (Task 4):** Create one of the three substantial UML Activity Diagrams.
- **Documentation (Task 1):** Co-create the complete UML class diagram drawn from scratch. (everyone adds own parts)
- **Tooling (Task 6):** Write a Dockerfile.
- **Project Admin (Task 1):** Document the identification of the GoF participants for Iterator, Composite, State and Decorator in your own design.
- **Project Admin (Task 6 & Section 5):** Write the short reflection in the PDF explaining how work was divided and integrated and the short team contribution statement.

All members:

- **Design (Task 1):** Write a concise description of the chosen domain alongside a short rationale for the important design and ownership decisions.